

SQL - Funkcje okna (Window functions)

Lab 2

Imiona i nazwiska: Natalia Bratek, Jakub Karczewski

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DDataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020
- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>

- <https://www.sqlservertutorial.net/sql-server-window-functions/>
 - <https://www.sqlshack.com/use-window-functions-sql-server/>
 - <https://www.postgresql.org/docs/current/tutorial-window.html>
 - <https://www.postgresqltutorial.com/postgresql-window-function/>
 - <https://www.sqlite.org/windowfunctions.html>
 - <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis ikon używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze "zmodyfikowanej wersji" bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, `row_number()`, `rank()`, `dense_rank()`

```
select productid, productname, unitprice, categoryid,  
       row_number() over(partition by categoryid order by unitprice desc) as rowno,  
       rank() over(partition by categoryid order by unitprice desc) as rankprice,  
       dense_rank() over(partition by categoryid order by unitprice desc) as denserankprice  
from products;
```

	ProductID	ProductName	UnitPrice	CategoryID	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263.5	1	1	1	1
2	43	Iphoh Coffee	46	1	2	2	2
3	2	Chang	19	1	3	3	3
4	1	Chai	18	1	4	4	4
5	35	Steeleye Stout	18	1	5	4	4
6	39	Chartreuse verte	18	1	6	4	4
7	76	Lakkalikööri	18	1	7	4	4
8	70	Outback Lager	15	1	8	8	5
9	34	Sasquatch Ale	14	1	9	9	6
10	67	Laughing Lumberjack Lager	14	1	10	9	6
11	75	Röhnbräu Klosterbier	7.75	1	11	11	7
12	24	Guinness Extra Stout	7.5	1	12	12	8

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje `row_number()`, `rank()`, `dense_rank()`. Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

Funkcja `row_number()` nadaje każdemu wierszowi kolejny numer, nawet jeśli wartości są takie same. `rank()` daje tę samą pozycję dla takich samych wartości, ale potem są przerwy w numeracji. `dense_rank()` działa podobnie do `rank()`, tylko że nie ma tych przerw i numeracja idzie po kolei.

```
select p.productid,
       p.productname,
       p.unitprice,
       p.categoryid,
       (select count(pp.productid) + 1 as c
        from products pp
        where (pp.categoryid = p.categoryid and pp.unitprice > p.unitprice)
              or (pp.categoryid = p.categoryid and pp.unitprice = p.unitprice and pp.productid < p.productid)) as rowno,
       (select count(pp.productid) + 1
        from products pp
        where pp.categoryid = p.categoryid
              and pp.unitprice > p.unitprice) as rankprice,
       (select count(t.c) + 1
        from (select distinct pp.unitprice as c
              from products pp
              where pp.categoryid = p.categoryid
                  and pp.unitprice > p.unitprice) t) as denserankprice
from products p
order by p.categoryid, p.unitprice desc, p.productid
```

	ProductID	ProductName	UnitPrice	CategoryID	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263.5	1	1	1	1
2	43	Iphoh Coffee	46	1	2	2	2
3	2	Chang	19	1	3	3	3
4	1	Chai	18	1	4	4	4
5	35	Steeleye Stout	18	1	5	4	4
6	39	Chartreuse verte	18	1	6	4	4
7	76	Lakkalikööri	18	1	7	4	4
8	78	Outback Lager	15	1	8	8	5
9	34	Sasquatch Ale	14	1	9	9	6
10	67	Laughing Lumberjack Lager	14	1	10	9	6
11	75	Röhnbräu Klosterbier	7.75	1	11	11	7
12	24	Guinness Extra Stout	7.5	1	12	12	8

Wniosek

- Zapytanie z funkcją okna ma koszt 6.11, używa jednego Seq Scan tabeli products, Sort i WindowAgg
- Wersja bez funkcji okna ma koszt 539.43, wykonuje cztery Seq Scan tabeli products
- Zapytanie z funkcją okna ma mniejszy koszt

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wynikowy powinien zawierać:

- rok
- id produktu
- nazwę produktu
- cenę
- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

Zapytanie z funkcją okna (row_number) dla MS SQL Server

```
with ranking as (  
    select  
        YEAR([date]) as year,  
        productid,  
        productname,  
        unitprice,  
        [date],  
        row_number() over (  
            partition by productid, YEAR([date])  
            order by unitprice desc  
        ) as rank_pos  
    from product_history  
)  
select *  
from ranking  
where rank_pos <= 4  
order by year, productid, rank_pos;
```

Zapytanie z funkcją okna (row_number) z ograniczeniem wierszy dla MS SQL Server

```
with ranking as (  
    select  
        YEAR([date]) as year,  
        productid,  
        productname,  
        unitprice,  
        [date],  
        row_number() over (  
            partition by productid, YEAR([date])  
            order by unitprice desc  
        ) as rank_pos  
    from product_history  
    where productid <= 10  
)  
select *  
from ranking  
where rank_pos <= 4  
order by year, productid, rank_pos;
```

Zapytanie bez funkcji okna z ograniczeniem wierszy dla MS SQL Server:

```
WITH base AS (  
    SELECT  
        productid,  
        productname,  
        unitprice,  
        [date],  
        YEAR([date]) AS rok,  
        id  
    FROM product_history  
    WHERE productid <= 10  
)  
, price_rank AS (  
    SELECT  
        b1.*,  
        COUNT(b2.unitprice) + 1 AS rn  
    FROM base b1  
    LEFT JOIN base b2  
        ON b1.productid = b2.productid  
        AND b1.rok = b2.rok
```

```

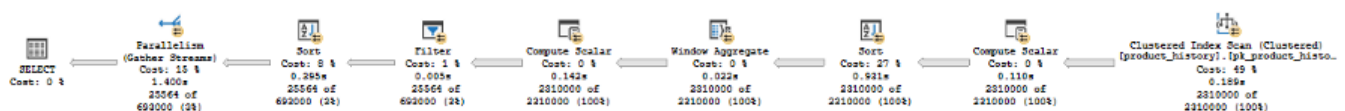
AND (
    b2.unitprice > b1.unitprice
OR (b2.unitprice = b1.unitprice AND b2.[date] < b1.[date]))
OR (b2.unitprice = b1.unitprice AND b2.[date] = b1.[date] AND b2.id < b1.id)
)
GROUP BY
    b1.productid,
    b1.productname,
    b1.unitprice,
    b1.[date],
    b1.rok,
    b1.id
)
SELECT *
FROM price_rank
WHERE rn <= 4
ORDER BY rok, productid, rn;

```

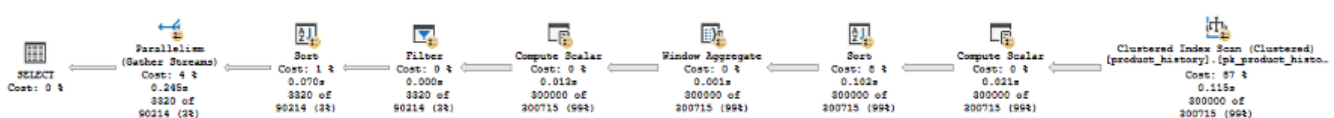
MS SQL Server

	year	productid	productname	unitprice	date	rank_pos
1	1940	1	Chai	27.98	1940-07-13	1
2	1940	1	Chai	27.95	1940-11-17	2
3	1940	1	Chai	27.92	1940-06-20	3
4	1940	1	Chai	27.87	1940-06-21	4
5	1940	2	Chang	28.98	1940-07-13	1
6	1940	2	Chang	28.95	1940-11-17	2
7	1940	2	Chang	28.92	1940-06-20	3
8	1940	2	Chang	28.86	1940-06-21	4
9	1940	3	Aniseed Syrup	19.99	1940-07-13	1
10	1940	3	Aniseed Syrup	19.97	1940-11-17	2
11	1940	3	Aniseed Syrup	19.96	1940-06-20	3
12	1940	3	Aniseed Syrup	19.93	1940-06-21	4
13	1940	4	Chef Anton's Cajun Seasoning	31.98	1940-07-13	1

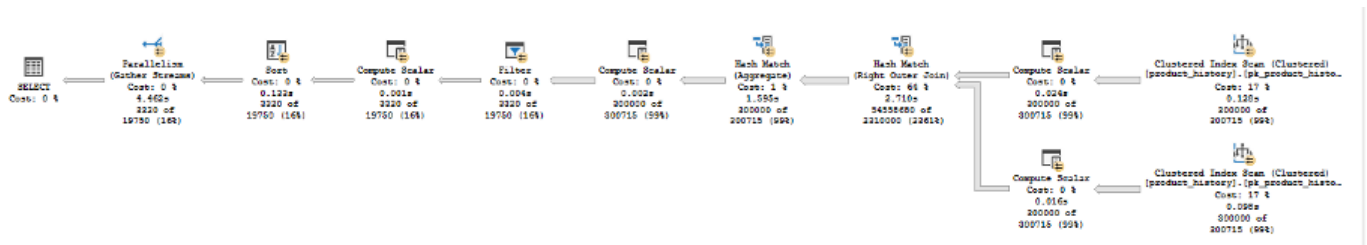
- pierwsze zapytanie



- drugie zapytanie



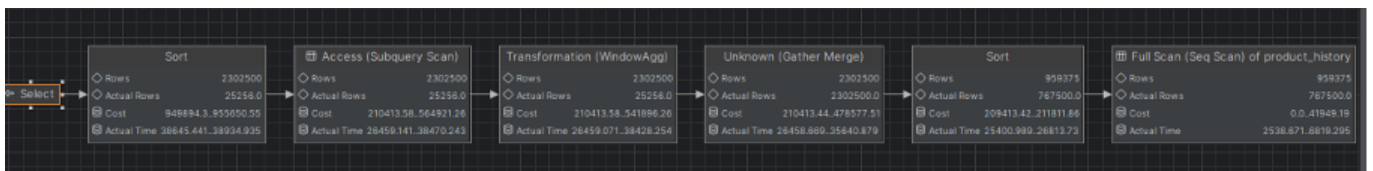
- trzecie zapytanie



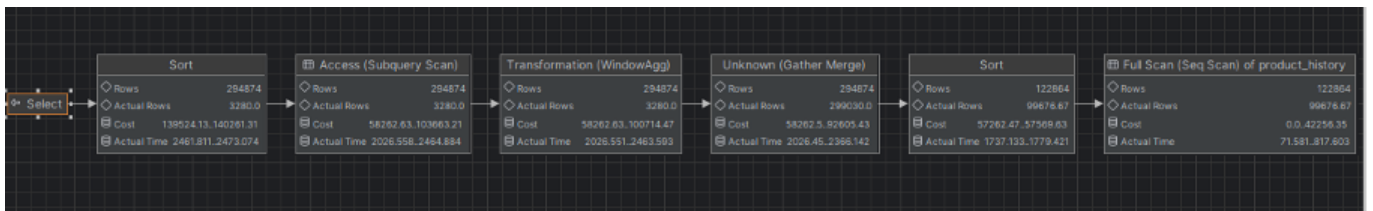
PostgreSQL

- pierwsze zapytanie

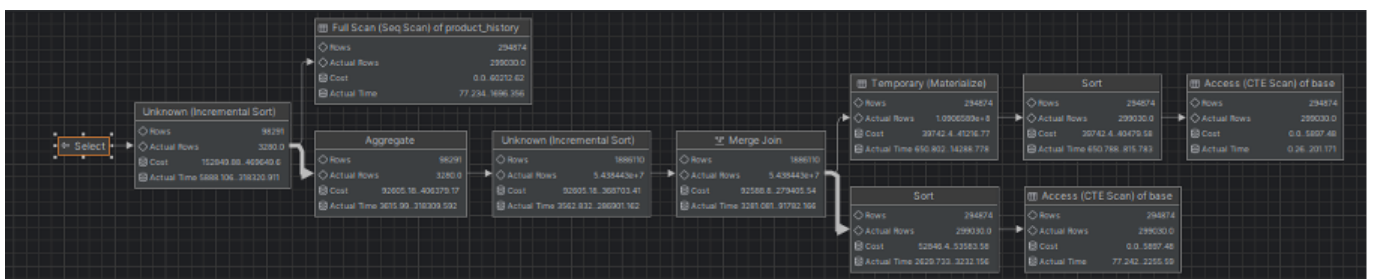
	year	productid	productname	unitprice	date	rank_pos
1	1940	1	Chai	27.98	1940-07-13	1
2	1940	1	Chai	27.95	1940-11-17	2
3	1940	1	Chai	27.92	1940-06-20	3
4	1940	1	Chai	27.87	1940-06-21	4
5	1940	2	Chang	28.98	1940-07-13	1
6	1940	2	Chang	28.95	1940-11-17	2
7	1940	2	Chang	28.92	1940-06-20	3
8	1940	2	Chang	28.86	1940-06-21	4
9	1940	3	Aniseed Syrup	19.99	1940-07-13	1
10	1940	3	Aniseed Syrup	19.97	1940-11-17	2



- drugie zapytanie



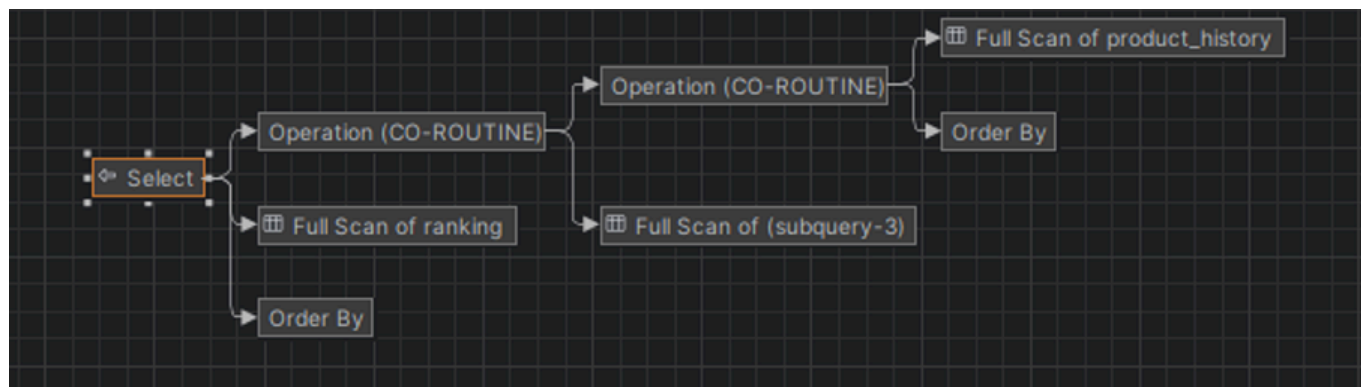
- trzecie zapytanie



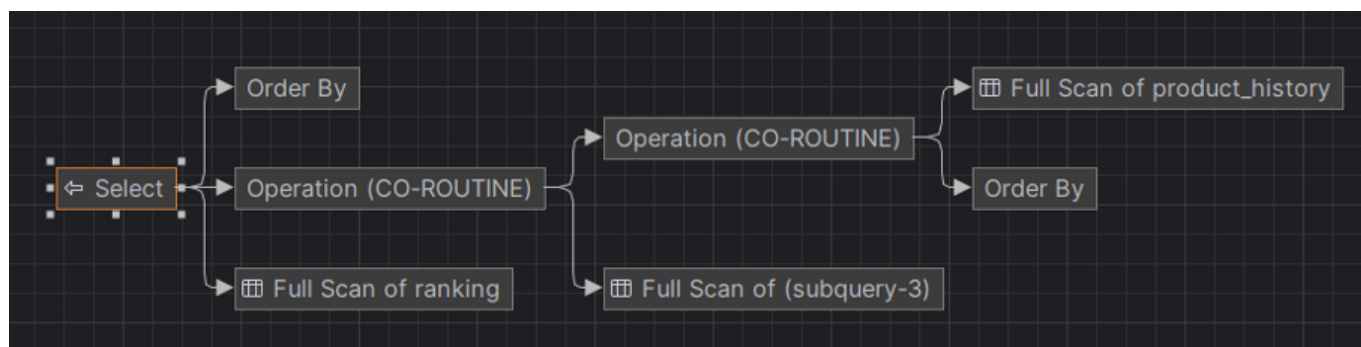
SQLite

- pierwsze zapytanie

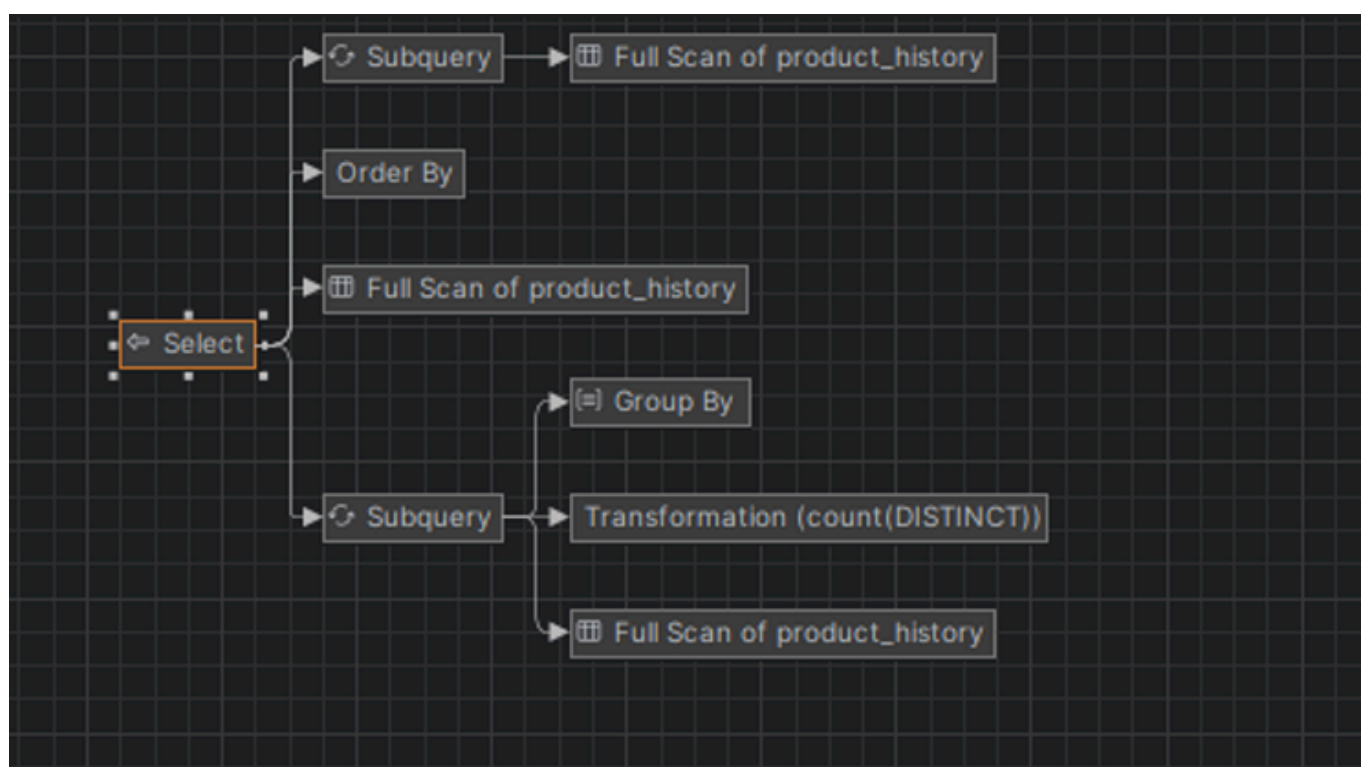
	year	productid	productname	unitprice	date	rank_pos
1	1940	1	Chai	27.98	1940-07-13	1
2	1940	1	Chai	27.95	1940-11-17	2
3	1940	1	Chai	27.92	1940-06-20	3
4	1940	1	Chai	27.87	1940-06-21	4
5	1940	2	Chang	28.98	1940-07-13	1
6	1940	2	Chang	28.95	1940-11-17	2
7	1940	2	Chang	28.92	1940-06-20	3



- drugie zapytanie



- trzecie zapytanie



Czasy zapytań

Baza danych	Funkcja okna	Funkcja okna z ograniczeniem	Bez funkcji okna z ograniczeniem
MS SQL Server	2 s	0 s	4 s
PostgreSQL	39 s	5 s	4 min 34 s
SQLite	21 s	2 s	9 min 32 s

Wnioski

MS SQL Server

- Zapytanie z funkcją okna ma Clustered Index Scan dla tabeli product_history , Window Aggregate i Sort. Skanuje całą tabelę, 2310000 wierszy
- Zapytanie z funkcją okna z ograniczeniem ma podobny plan, ale przetwarza mniej wierszy (300000)
- Zapytanie bez funkcji okna ma dwa Clustered Index Scan i Hash Match
- Funkcja okna ma prostszy plan niż wersja bez okna

PostgreSQL

- Zapytanie z funkcją okna ma koszt 955055.55, używa Seq Scan, Sort i WindowAgg
- Zapytanie z funkcją okna z ograniczeniem ma koszt 140261.31, dużo mniej niż wersja bez ograniczenia
- Zapytanie bez funkcji okna ma koszt 459640.6, ma Merge Join i Aggregate
- Zapytanie bez funkcji okna ma dużo większy koszt niż funkcja okna z ograniczeniem

SQLite

SQLite

- Pierwsze i drugie zapytanie (z funkcją okna) mają taki sam plan, m.in. Full Scan tabeli product_history i ranking
- Trzecie zapytanie (bez funkcji okna) ma trzy Full Scan tabeli product_history
- Wersja bez funkcji okna ma dużo więcej operacji i skanów tabeli
- Funkcja okna ma prostszy plan w MS SQL Server, PostgreSQL, SQLite

Zadanie 3

Funkcje `lag()`, `lead()`

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `lag()`, `lead()`

```

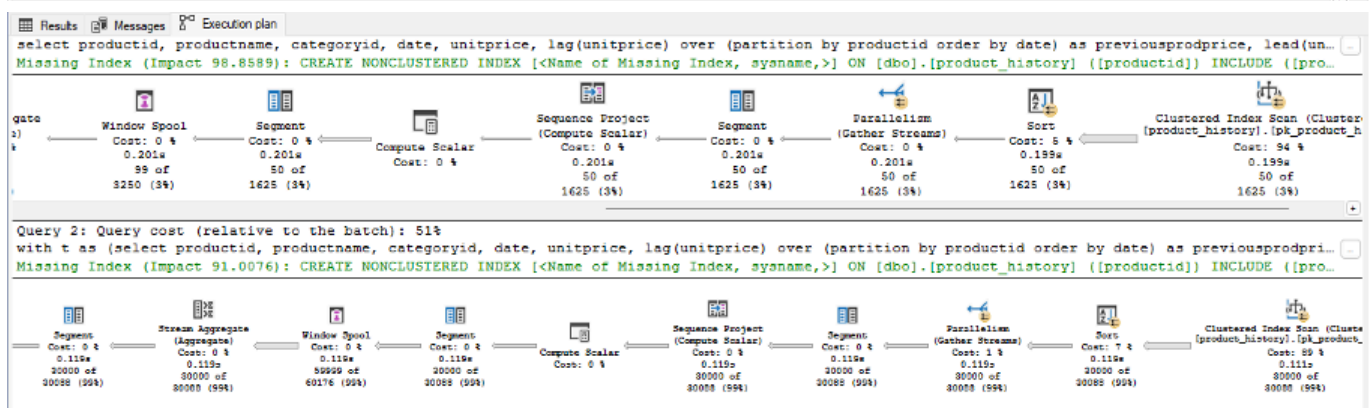
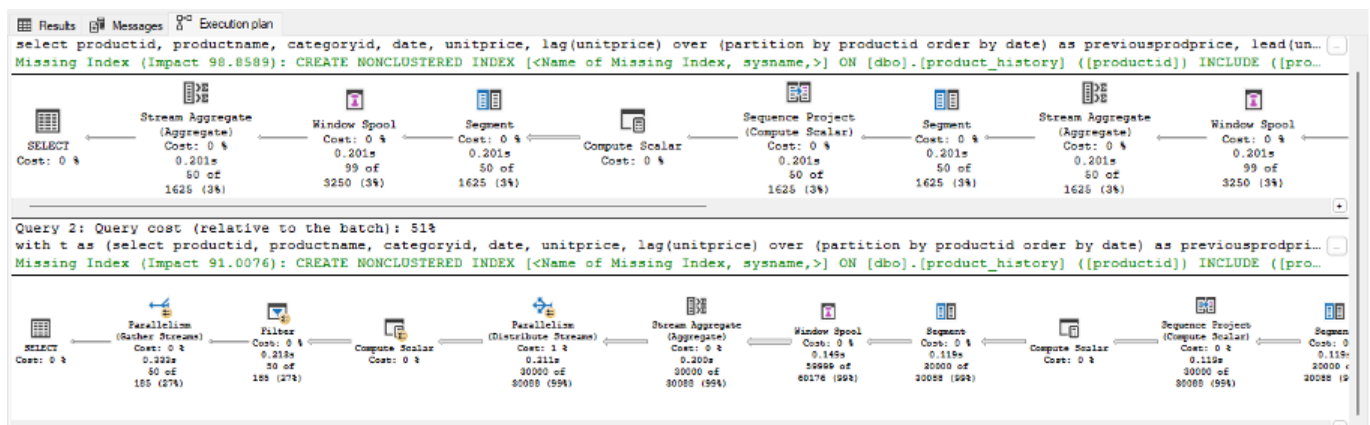
select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date) as previousprodprice,
       lead(unitprice) over (partition by productid order by date) as nextprodprice
from product_history
where productid = 1 and year(date) = 2022
order by date;

with t as (select productid, productname, categoryid, date, unitprice,
       lag(unitprice) over (partition by productid order by date) as previousprodprice,
       lead(unitprice) over (partition by productid order by date) as nextprodprice
       from product_history)
select * from t
where productid = 1 and year(date) = 2022
order by date;

```

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	25.31	NULL	20.69
2	1	Chai	1	2022-01-02	20.69	25.31	18.16
3	1	Chai	1	2022-01-03	18.16	20.69	23.93
4	1	Chai	1	2022-01-04	23.93	18.16	13.69
5	1	Chai	1	2022-01-05	13.69	23.93	25.34
6	1	Chai	1	2022-01-06	25.34	13.69	17.89
7	1	Chai	1	2022-01-07	17.89	25.34	17.12
8	1	Chai	1	2022-01-08	17.12	17.89	26.73

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	25.31	21.10	20.69
2	1	Chai	1	2022-01-02	20.69	25.31	18.16
3	1	Chai	1	2022-01-03	18.16	20.69	23.93
4	1	Chai	1	2022-01-04	23.93	18.16	13.69
5	1	Chai	1	2022-01-05	13.69	23.93	25.34
6	1	Chai	1	2022-01-06	25.34	13.69	17.89



Jak działają funkcje `lag()`, `lead()`?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

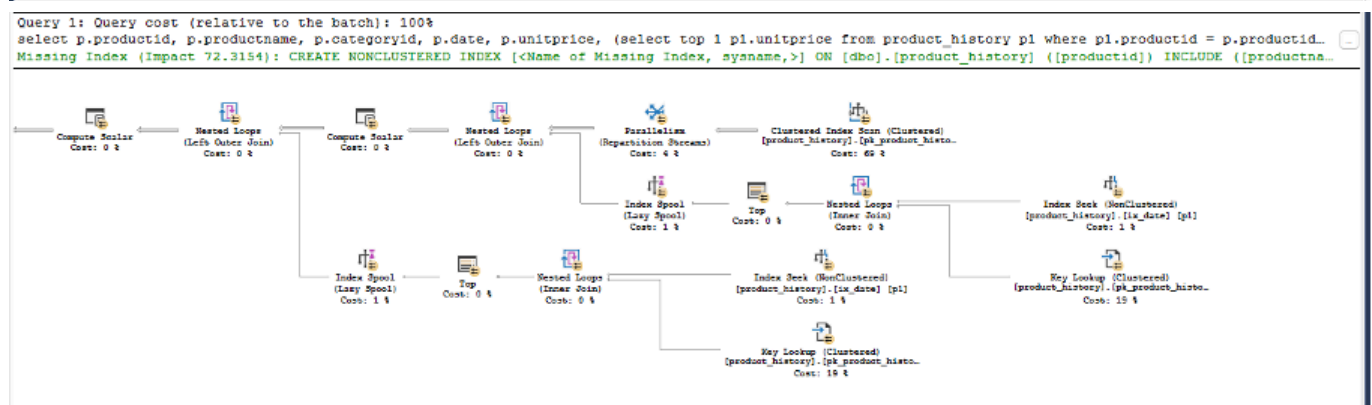
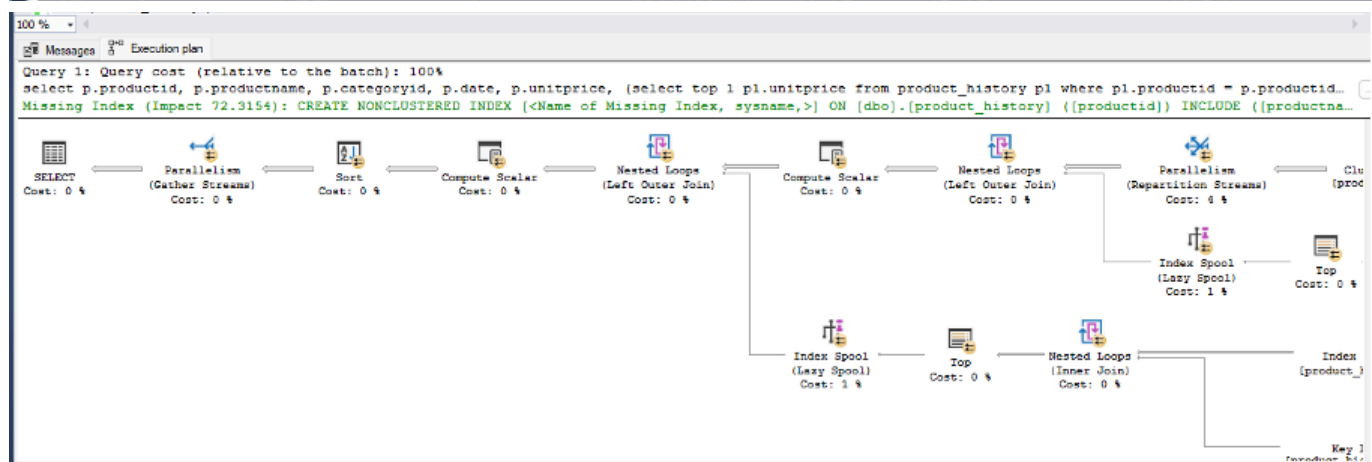
Funkcja `lag()` zwraca wartość z poprzedniego wiersza, a dla pierwszego wiersza wartość wynosi NULL (brak poprzedniego). Funkcja `lead()` zwraca wartość z następnego wiersza, a dla ostatniego wiersza wartość wynosi NULL (brak następnego).

```
select p.productid,
       p.productname,
       p.categoryid,
       p.date,
       p.unitprice,
       (select top 1 p1.unitprice
        from product_history p1
        where p1.productid = p.productid and year(p1.date) = 2022 and p1.date < p.date
        order by p1.date desc) as previousprodprice,
       (select top 1 p1.unitprice
        from product_history p1
        where p1.productid = p.productid and year(p1.date) = 2022 and p1.date > p.date
        order by p1.date asc) as nextprodprice
from product_history p
where p.productid = 1
     and year(p.date) = 2022
order by p.date;
```

100 %

Results Messages

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	25.31	NULL	20.69
2	1	Chai	1	2022-01-02	20.69	25.31	18.16
3	1	Chai	1	2022-01-03	18.16	20.69	23.93
4	1	Chai	1	2022-01-04	23.93	18.16	13.69
5	1	Chai	1	2022-01-05	13.69	23.93	25.34
6	1	Chai	1	2022-01-06	25.34	13.69	17.89
7	1	Chai	1	2022-01-07	17.89	25.34	17.12
8	1	Chai	1	2022-01-08	17.12	17.89	26.73
9	1	Chai	1	2022-01-09	26.73	17.12	19.50
10	1	Chai	1	2022-01-10	19.50	26.73	10.01
11	1	Chai	1	2022-01-11	10.01	19.50	20.25
12	1	Chai	1	2022-01-12	20.25	10.01	15.67
13	1	Chai	1	2022-01-13	15.67	20.25	22.22
14	1	Chai	1	2022-01-14	22.22	15.67	16.41
15	1	Chai	1	2022-01-15	16.41	22.22	17.83
16	1	Chai	1	2022-01-16	17.83	16.41	25.14
17	1	Chai	1	2022-01-17	25.14	17.83	21.15



Wnioski

- Funkcja okna ma jeden Clustered Index Scan tabeli `product_history`, używa Window Spool, Segment i Sequence Project.

- W obu poleceniach z funkcją okna jest po jednym Clustered Index Scan tabeli product_history
- Wersja bez funkcji okna ma Clustered Index Scan, dwa Index Seek
- Wersja bez funkcji okna ma dużo więcej operacji, funkcja okna ma prostszy plan

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta, nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

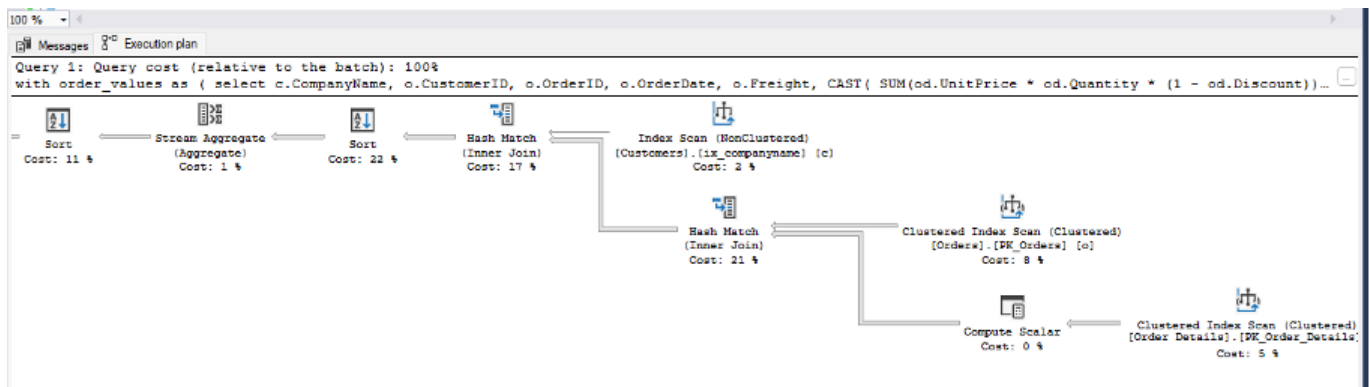
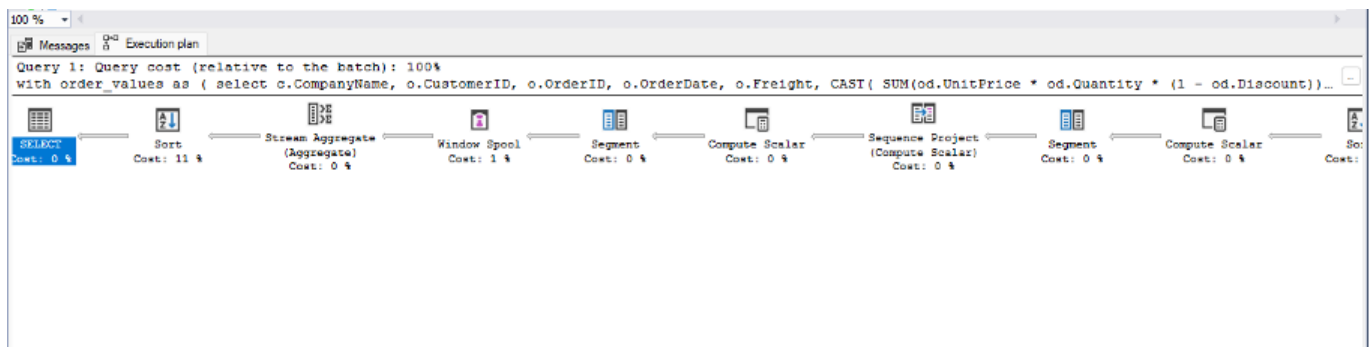
Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

Wyniki:

```
with order_values as (  
    select  
        c.CompanyName,  
        o.CustomerID,  
        o.OrderID,  
        o.OrderDate,  
        o.Freight,  
        CAST(  
            SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) + o.Freight AS decimal(10,2)) as order_total  
        from Orders o  
        join [Order Details] od on o.OrderID = od.OrderID  
        join Customers c on o.CustomerID = c.CustomerID  
        group by c.CompanyName, o.CustomerID, o.OrderID, o.OrderDate, o.Freight)  
select  
    CompanyName,  
    OrderID,  
    OrderDate,  
    order_total,  
    lag(OrderID) over (partition by CustomerID order by OrderDate) as prev_order_id,  
    lag(OrderDate) over (partition by CustomerID order by OrderDate) as prev_order_date,  
    lag(order_total) over (partition by CustomerID order by OrderDate) as prev_order_value  
from order_values  
order by CompanyName, OrderDate;
```

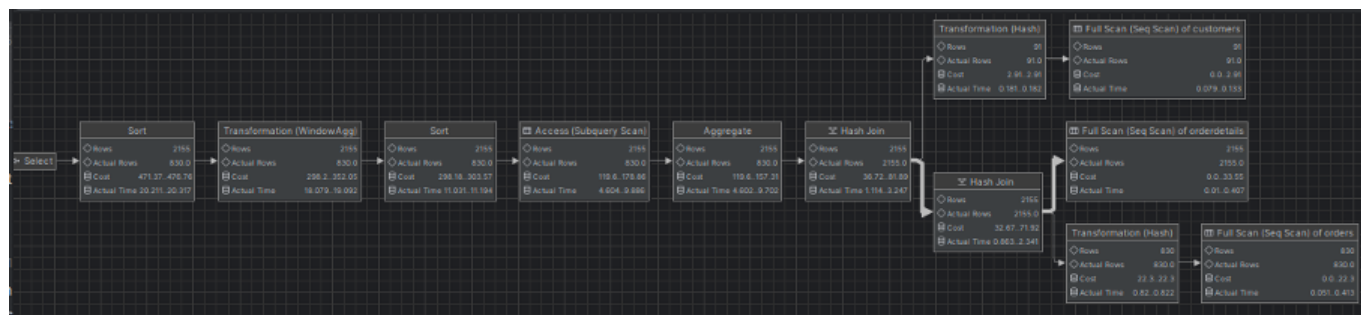
- MS SQL SERVER

	CompanyName	OrderID	OrderDate	order_total	prev_order_id	prev_order_date	prev_order_value
1	Alfreds Futterkiste	10643	1997-08-25 00:00:00.000	843.96	NULL	NULL	NULL
2	Alfreds Futterkiste	10692	1997-10-03 00:00:00.000	939.02	10643	1997-08-25 00:00:00.000	843.96
3	Alfreds Futterkiste	10702	1997-10-13 00:00:00.000	353.94	10692	1997-10-03 00:00:00.000	939.02
4	Alfreds Futterkiste	10835	1998-01-15 00:00:00.000	915.33	10702	1997-10-13 00:00:00.000	353.94
5	Alfreds Futterkiste	10952	1998-03-16 00:00:00.000	511.62	10835	1998-01-15 00:00:00.000	915.33
6	Alfreds Futterkiste	11011	1998-04-09 00:00:00.000	934.71	10952	1998-03-16 00:00:00.000	511.62
7	Ana Trujillo Emparedados y helados	10308	1996-09-18 00:00:00.000	90.41	NULL	NULL	NULL
8	Ana Trujillo Emparedados y helados	10625	1997-08-08 00:00:00.000	523.65	10308	1996-09-18 00:00:00.000	90.41
9	Ana Trujillo Emparedados y helados	10759	1997-11-28 00:00:00.000	331.99	10625	1997-08-08 00:00:00.000	523.65
10	Ana Trujillo Emparedados y helados	10926	1998-03-04 00:00:00.000	554.32	10759	1997-11-28 00:00:00.000	331.99
11	Antonio Moreno Taquería	10365	1996-11-27 00:00:00.000	425.20	NULL	NULL	NULL
12	Antonio Moreno Taquería	10507	1997-04-15 00:00:00.000	796.51	10365	1996-11-27 00:00:00.000	425.20
13	Antonio Moreno Taquería	10535	1997-05-13 00:00:00.000	1956.49	10507	1997-04-15 00:00:00.000	796.51
14	Antonio Moreno Taquería	10573	1997-06-19 00:00:00.000	2166.84	10535	1997-05-13 00:00:00.000	1956.49
15	Antonio Moreno Taquería	10677	1997-09-22 00:00:00.000	817.40	10573	1997-06-19 00:00:00.000	2166.84
16	Antonio Moreno Taquería	10682	1997-09-25 00:00:00.000	411.63	10677	1997-09-22 00:00:00.000	817.40
17	Antonio Moreno Taquería	10856	1998-01-28 00:00:00.000	718.43	10682	1997-09-25 00:00:00.000	411.63
18	Around the Hom	10355	1996-11-15 00:00:00.000	521.95	NULL	NULL	NULL
19	Around the Hom	10383	1996-12-16 00:00:00.000	933.24	10355	1996-11-15 00:00:00.000	521.95
20	Around the Hom	10452	1997-02-21 00:00:00.000	422.06	10383	1996-12-16 00:00:00.000	933.24



- PostgreSQL

companyname	orderid	orderdate	order_total	prev_order_id	prev_order_date	prev_order_value
1 Alfreds Futterkiste	10643	1997-08-25	843.96	<null>	<null>	<null>
2 Alfreds Futterkiste	10692	1997-10-03	939.02	10643	1997-08-25	843.96
3 Alfreds Futterkiste	10702	1997-10-13	353.94	10692	1997-10-03	939.02
4 Alfreds Futterkiste	10835	1998-01-15	915.33	10702	1997-10-13	353.94
5 Alfreds Futterkiste	10952	1998-03-16	511.62	10835	1998-01-15	915.33
6 Alfreds Futterkiste	11011	1998-04-09	934.71	10952	1998-03-16	511.62
7 Ana Trujillo Emparedados y helados	10308	1996-09-18	90.41	<null>	<null>	<null>
8 Ana Trujillo Emparedados y helados	10625	1997-08-08	523.65	10308	1996-09-18	90.41
9 Ana Trujillo Emparedados y helados	10759	1997-11-28	331.99	10625	1997-08-08	523.65
10 Ana Trujillo Emparedados y helados	10926	1998-03-04	554.32	10759	1997-11-28	331.99
11 Antonio Moreno Taquería	10365	1996-11-27	425.20	<null>	<null>	<null>
12 Antonio Moreno Taquería	10507	1997-04-15	796.51	10365	1996-11-27	425.2
13 Antonio Moreno Taquería	10535	1997-05-13	1956.49	10507	1997-04-15	796.51
14 Antonio Moreno Taquería	10573	1997-06-19	2166.84	10535	1997-05-13	1956.49



Wnioski

MS SQL Server

- Występuje jeden skan każdej tabeli: Clustered Index Scan dla Orders, Order Details, Index Scan dla Customers
- Są dwa Hash Match (Inner Join) do połączenia tabel

PostgreSQL

- Jest jeden Seq Scan każdej tabeli: customers, orders, orderdetails
- Występują dwa Hash Join do połączenia tabel
- Funkcja okna jest realizowana przez Transformation (WindowAgg)
- Koszt wynosi 475.76

Zadanie 5

Funkcje `first_value()`, `last_value()`

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje `first_value()`, `last_value()`.

Skomentuj uzyskane wyniki. Czy funkcja `first_value` pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja `last_value()` pokazuje najtańszy produkt?

Co jest przyczyną takiego działania funkcji `last_value`.

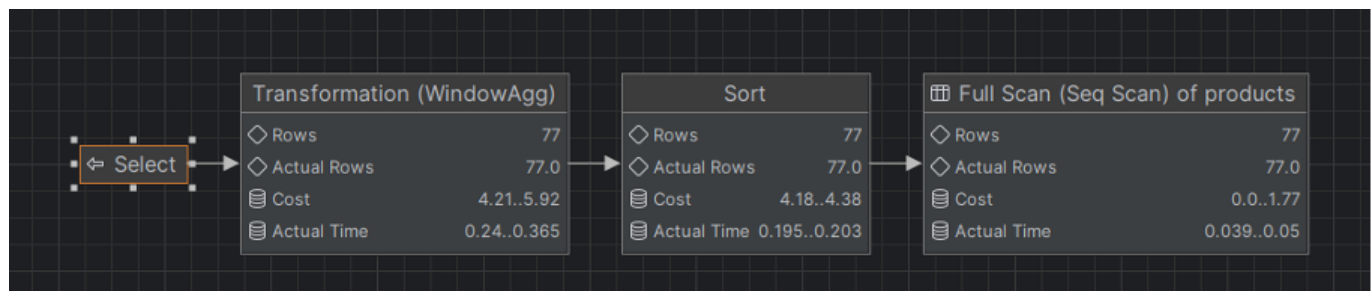
Co trzeba zmienić żeby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

```
select productid, productname, unitprice, categoryid,
       first_value(productname) over (partition by categoryid
                                     order by unitprice desc) first,
       last_value(productname) over (partition by categoryid
                                    order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```


Wyniki:

	ProductID	ProductName	UnitPrice	CategoryID	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Côte de Blaye
2	43	Ipoh Coffee	46	1	Côte de Blaye	Ipoh Coffee
3	2	Chang	19	1	Côte de Blaye	Chang
4	1	Chai	18	1	Côte de Blaye	Lakkalikööri
5	35	Steeleye Stout	18	1	Côte de Blaye	Lakkalikööri
6	39	Chartreuse verte	18	1	Côte de Blaye	Lakkalikööri
7	76	Lakkalikööri	18	1	Côte de Blaye	Lakkalikööri
8	70	Outback Lager	15	1	Côte de Blaye	Outback Lager
9	34	Sasquatch Ale	14	1	Côte de Blaye	Laughing Lumberjack Lager
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Laughing Lumberjack Lager
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Rhönbräu Klosterbier
12	24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
13	63	Verie-spread	43.9	2	Verie-spread	Verie-spread



`first_value` bierze wartość z pierwszego wiersza w danym oknie, czyli zwraca najdroższy produkt w danej kategorii.

`last_value` bierze wartość z ostatniego wiersza w danym oknie.

W przypadku funkcji `last_value` - okno jest definiowane jako "od początku do aktualnego wiersza". Dlatego funkcja `last_value` nie pokazuje najtańszego produktu, tylko pokazuje produkt z aktualnego wiersza, czyli najtańszy produkt w danej chwili, bez uwzględnienia wierszy, które są po nim.

Aby funkcja `last_value` pokazywała najtańszy produkt w danej kategorii, trzeba rozszerzyć okno, tak aby obejmowało wszystkie wiersze w danej kategorii. Można to zrobić za pomocą klauzuli `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Poprawione zapytanie:

```

select productid, productname, unitprice, categoryid,
       first_value(productname) over (
         partition by categoryid
         order by unitprice desc
       ) as first,
       last_value(productname) over (
         partition by categoryid
         order by unitprice desc
         rows between unbounded preceding and unbounded following
       ) as last
from products
order by categoryid, unitprice desc;
  
```

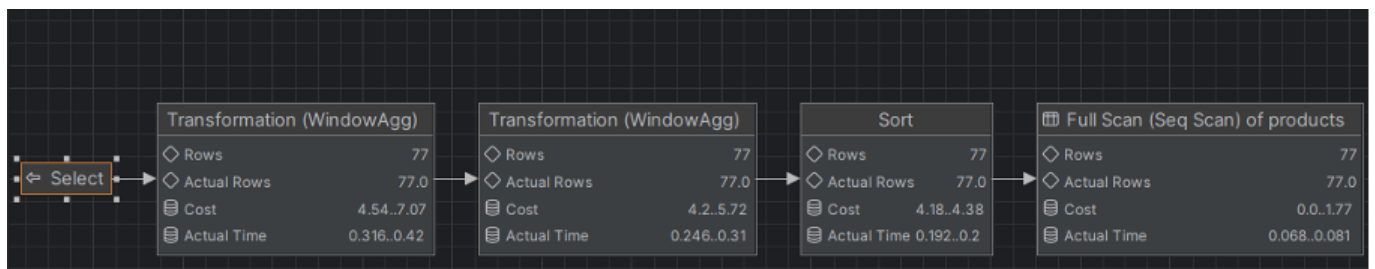
Wynik zapytania z poprawionym `last_value`:

```
1 select productid, productname, unitprice, categoryid,
2     first_value(productname) over (
3         partition by categoryid
4         order by unitprice desc
5     ) as first,
6     last_value(productname) over (
7         partition by categoryid
8         order by unitprice desc
9         rows between unbounded preceding and unbounded following
10    ) as last
11 from products
12 order by categoryid, unitprice desc;
```

Output Result 1

ProductID	ProductName	UnitPrice	CategoryID	first	last
38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
43	Ipoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica
67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Guaraná Fantástica
75	Rhönbräu Klosterbier		1	Côte de Blaye	Guaraná Fantástica

77 rows



```
select productid, productname, unitprice, categoryid,
       first_value(productname) over (
         partition by categoryid
         order by unitprice desc
       ) as first,
       last_value(productname) over (
         partition by categoryid
         order by unitprice desc
         rows between current row and unbounded following
       ) as last
from products
order by categoryid, unitprice desc;
```

Session Result 3 x

77 rows

	productid	productname	unitprice	categoryid	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
2	43	Iphoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
3	2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
4	1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
5	76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
6	35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
7	39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
8	70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
9	34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica

Operation	Rows	Actual Rows	Cost	Actual Time
Select	77	77.0	4.54..7.07	0.172..0.287
Transformation (WindowAgg)	77	77.0	4.2..5.72	0.153..0.235
Transformation (WindowAgg)	77	77.0	4.18..4.38	0.14..0.146
Sort	77	77.0	0.0..1.77	0.09..0.101
Full Scan (Seq Scan) of products	77	77.0	0.0..1.77	0.09..0.101

Wnioski

- Wersja bez rows between ma koszt 5.92, używa jednego Seq Scan, Sort i jednego WindowAgg
- Wersja z rows between unbounded preceding and unbounded following ma koszt 7.07 i używa dwóch WindowAgg
- Wersja z rows between current row and unbounded following ma koszt 7.07, również używa dwóch WindowAgg
- Wersja bez rows between ma najmniejszy koszt
- We wszystkich wersjach jest tylko jeden Seq Scan tabeli products

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu

- nr zamówienia o najniższej wartości w danym miesiącu
- datę tego zamówienia
- wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

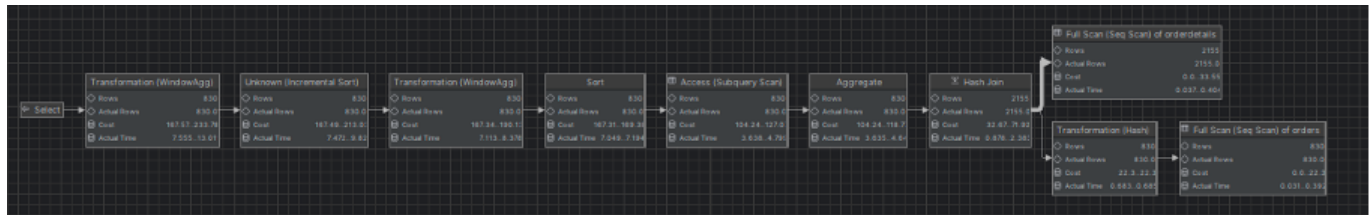
Wyniki:

```
WITH order_values AS (
  SELECT
    o.customerid,
    o.orderid,
    o.orderdate,
    SUM(od.unitprice * od.quantity * (1 - COALESCE(od.discount, 0)))
      + COALESCE(o.freight, 0) AS order_value,
    to_char(o.orderdate, 'YYYY-MM') AS year_month
  FROM orders o
  JOIN "Order Details" od
    ON o.orderid = od.orderid
  GROUP BY
    o.customerid,
    o.orderid,
    o.orderdate,
    o.freight
)
SELECT
  customerid,
  orderid,
  orderdate,
  order_value,
  -- MIN
  FIRST_VALUE(orderid) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value ASC
  ) AS min_order_id,
  FIRST_VALUE(orderdate) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value ASC
  ) AS min_order_date,
  FIRST_VALUE(order_value) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value ASC
  ) AS min_order_value,
  -- MAX
  FIRST_VALUE(orderid) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value DESC
  ) AS max_order_id,
  FIRST_VALUE(orderdate) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value DESC
  ) AS max_order_date,
  FIRST_VALUE(order_value) OVER (
    PARTITION BY customerid, year_month
    ORDER BY order_value DESC
  ) AS max_order_value
FROM order_values
ORDER BY customerid, year_month, order_value;
```

	customerid	orderid	orderdate	order_value	min_order_id	min_order_date	min_order_value	max_order_
1	ALFKI	10643	1997-08-25	843.959981918335	10643	1997-08-25	843.959981918335	
2	ALFKI	10702	1997-10-13	353.9400005340576	10702	1997-10-13	353.9400005340576	
3	ALFKI	10692	1997-10-03	939.0200309753418	10702	1997-10-13	353.9400005340576	
4	ALFKI	10835	1998-01-15	915.3299987018108	10835	1998-01-15	915.3299987018108	
5	ALFKI	10952	1998-03-16	511.6199948191643	10952	1998-03-16	511.6199948191643	
6	ALFKI	11011	1998-04-09	934.7099996432662	11011	1998-04-09	934.7099996432662	
7	ANATR	10308	1996-09-18	90.40999925136566	10308	1996-09-18	90.40999925136566	
8	ANATR	10625	1997-08-08	523.6500015258789	10625	1997-08-08	523.6500015258789	
9	ANATR	10759	1997-11-28	331.98999977111816	10759	1997-11-28	331.98999977111816	
10	ANATR	10926	1998-03-04	554.3199892044067	10926	1998-03-04	554.3199892044067	
11	ANTON	10365	1996-11-27	425.1999816894531	10365	1996-11-27	425.1999816894531	
12	ANTON	10507	1997-04-15	796.5124955102801	10507	1997-04-15	796.5124955102801	
13	ANTON	10535	1997-05-13	1956.48999369666	10535	1997-05-13	1956.48999369666	

	customerid	orderid	orderdate	order_value	min_order_id	min_order_date	min_order_value	max_order_
1	ALFKI	10643	1997-08-25	843.959981918335	10643	1997-08-25	843.959981918335	
2	ALFKI	10702	1997-10-13	353.9400005340576	10702	1997-10-13	353.9400005340576	
3	ALFKI	10692	1997-10-03	939.0200309753418	10702	1997-10-13	353.9400005340576	
4	ALFKI	10835	1998-01-15	915.3299987018108	10835	1998-01-15	915.3299987018108	
5	ALFKI	10952	1998-03-16	511.6199948191643	10952	1998-03-16	511.6199948191643	
6	ALFKI	11011	1998-04-09	934.7099996432662	11011	1998-04-09	934.7099996432662	
7	ANATR	10308	1996-09-18	90.40999925136566	10308	1996-09-18	90.40999925136566	
8	ANATR	10625	1997-08-08	523.6500015258789	10625	1997-08-08	523.6500015258789	
9	ANATR	10759	1997-11-28	331.98999977111816	10759	1997-11-28	331.98999977111816	
10	ANATR	10926	1998-03-04	554.3199892044067	10926	1998-03-04	554.3199892044067	
11	ANTON	10365	1996-11-27	425.1999816894531	10365	1996-11-27	425.1999816894531	
12	ANTON	10507	1997-04-15	796.5124955102801	10507	1997-04-15	796.5124955102801	
13	ANTON	10535	1997-05-13	1956.48999369666	10535	1997-05-13	1956.48999369666	

Plan zapytania



Wnioski

- Plan ma jeden Seq Scan tabel orders i orderdetails, połączone przez Hash Join
- Są dwa WindowAgg: jeden dla wartości MIN (ASC), drugi dla MAX (DESC)
- Koszt zapytania to 233.71

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

Kod SQL z funkcją okna:

```
with base as (  
    select  
        id,  
        productid,  
        [date],  
        value * quantity as sales_value  
    from product_history  
)  
select TOP 100  
    *,  
    sum(sales_value) over (  
        partition by productid, format([date], 'yyyy-MM')  
        order by [date]  
    ) as cumulative_sales  
from base;
```

Kod SQL bez funkcji okna:

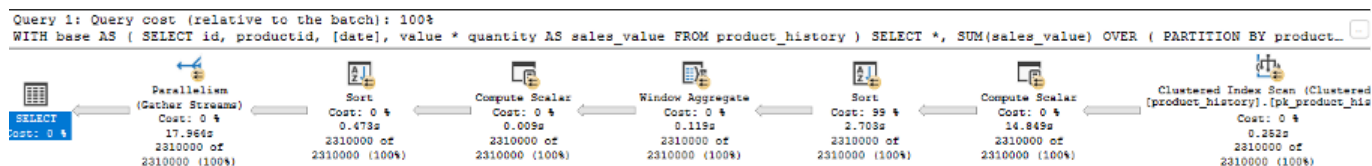
```
select TOP 100  
    p1.id,  
    p1.productid,  
    p1.[date],  
    p1.value * p1.quantity as sales_value,  
    (select sum(p2.value * p2.quantity)  
     from product_history p2  
     where p2.productid = p1.productid  
           and year(p2.[date]) = year(p1.[date])  
           and month(p2.[date]) = month(p1.[date])  
           and p2.[date] <= p1.[date]  
    ) as cumulative_sales  
from product_history p1
```


Baza danych	Funkcja okna	Bez funkcji okna
MS SQL Server	44 s	1 min 57 s
PostgreSQL	17 s	2 min 9 s
SQLite	1,7 s	1 min 28 s

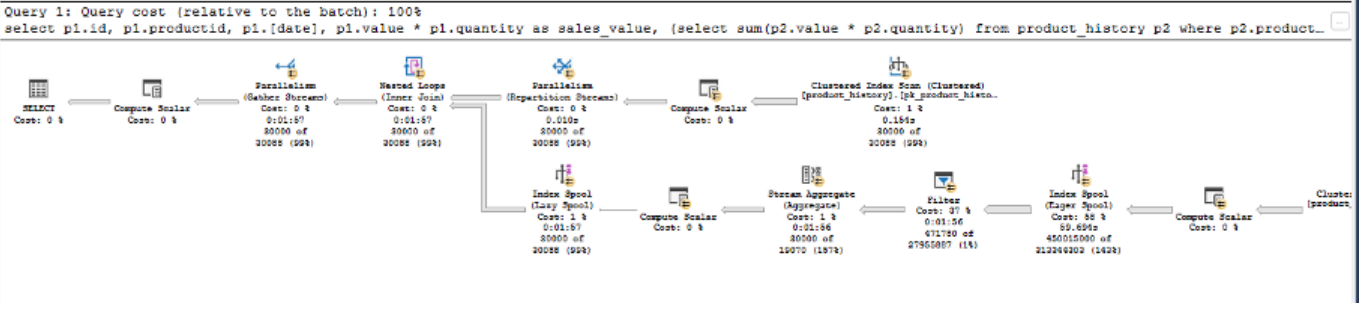
MS SQL Server

	id	productid	date	sales_value	cumulative_sales
1	1	1	1940-01-02	1585.00	1585.00
2	78	1	1940-01-03	2567.00	4152.00
3	155	1	1940-01-04	2742.00	6894.00
4	232	1	1940-01-05	1403.00	8297.00
5	309	1	1940-01-06	2658.00	10955.00
6	386	1	1940-01-07	2025.00	12980.00
7	463	1	1940-01-08	2123.00	15103.00
8	540	1	1940-01-09	1874.00	16977.00
9	617	1	1940-01-10	2331.00	19308.00
10	694	1	1940-01-11	2102.00	21410.00
11	771	1	1940-01-12	1787.00	23197.00

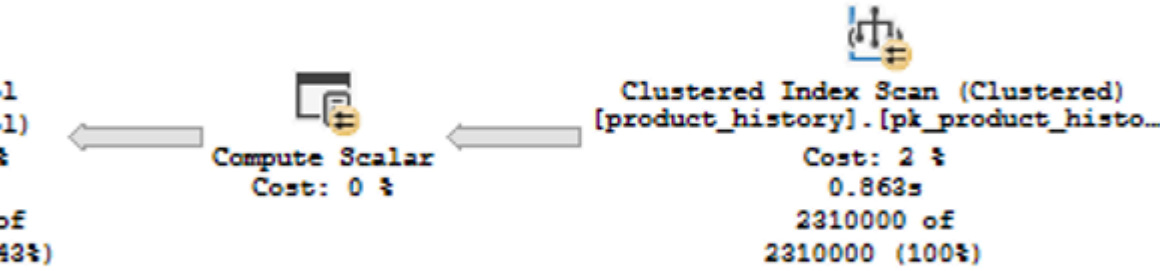
- zapytanie pierwsze



- zapytanie drugie



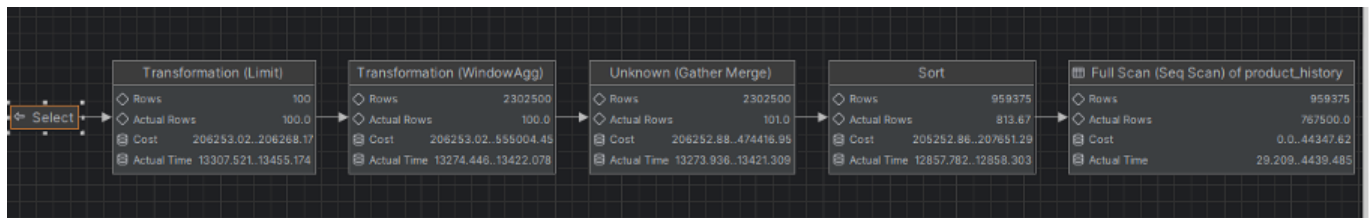
ity) from product_history p2 where p2.product...



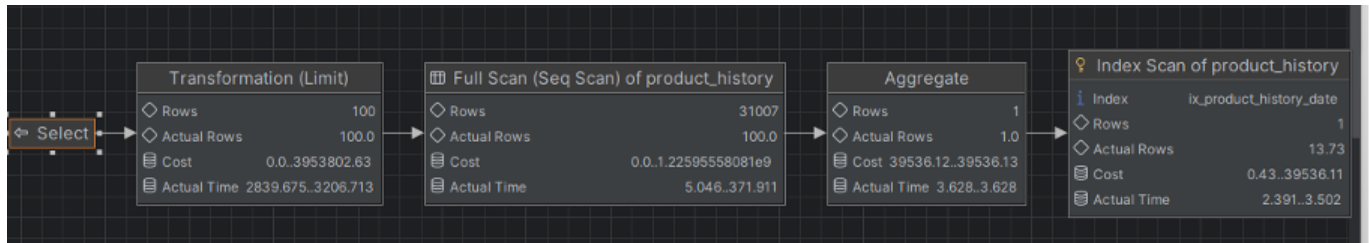
PostgreSQL

	id	productid	date	sales_value	cumulative_sales
1	3312	1	1940-02-14	1909	1909
2	3389	1	1940-02-15	1385	3294
3	3466	1	1940-02-16	1219	4513
4	3543	1	1940-02-17	1437	5950
5	3620	1	1940-02-18	1044	6994
6	3697	1	1940-02-19	1645	8639
7	3774	1	1940-02-20	1001	9640

- zapytanie pierwsze



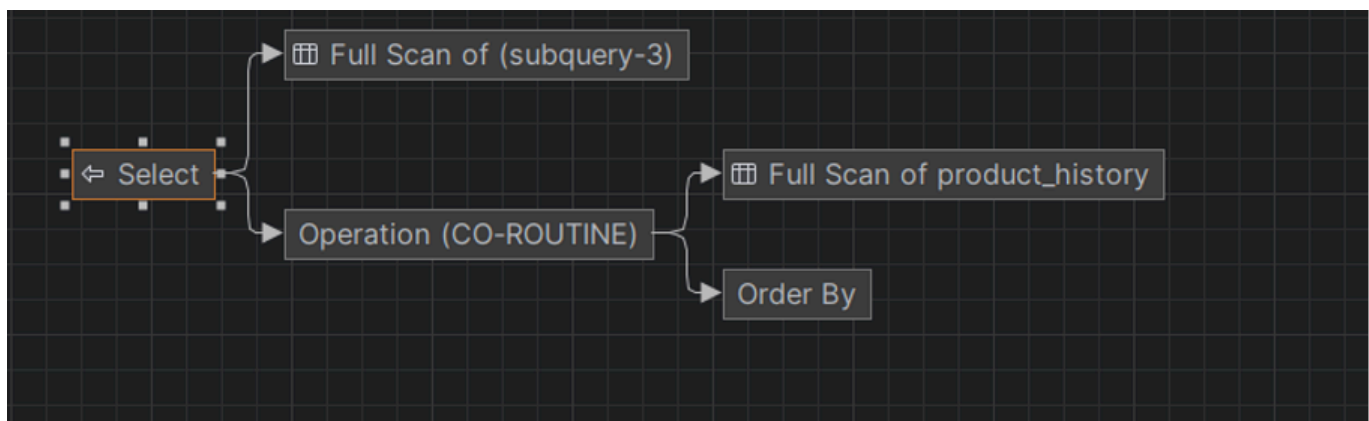
- zapytanie drugie



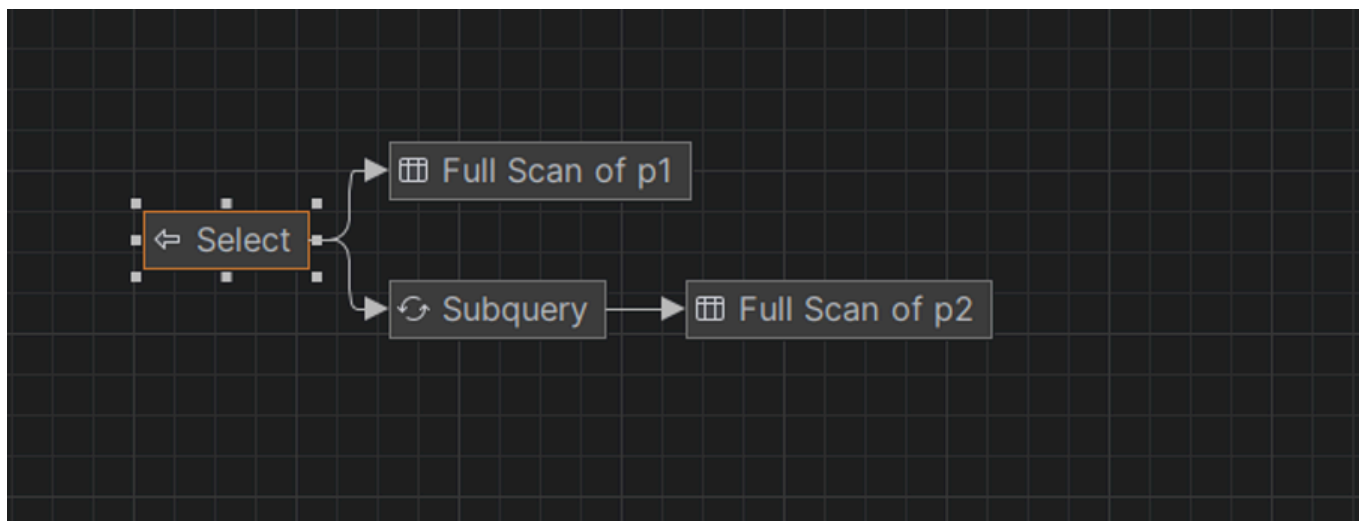
SQLite

	id	productid	date	sales_value	cumulative_sales
1	1	1	1940-01-02	1585	1585
2	78	1	1940-01-03	2567	4152
3	155	1	1940-01-04	2742	6894
4	232	1	1940-01-05	1403	8297
5	309	1	1940-01-06	2658	10955
6	386	1	1940-01-07	2025	12980
7	463	1	1940-01-08	2123	15103
8	540	1	1940-01-09	1874	16977
9	617	1	1940-01-10	2331	19308
10	694	1	1940-01-11	2102	21410
11	771	1	1940-01-12	1787	23197

- zapytanie pierwsze



- zapytanie drugie



Wnioski

MS SQL Server

- Zapytanie z funkcją okna ma jeden Clustered Index Scan dla tabeli product_history, Window Aggregate i Sort
- Zapytanie bez funkcji okna ma dwa Clustered Index Scan
- Plan zapytania z wersją z funkcją okna jest prostsza

PostgreSQL

- Zapytanie z funkcją okna ma koszt 206268.17, używa jednego Seq Scan, Sort i WindowAgg
- Zapytanie bez funkcji okna ma koszt 3953802.63 (prawie 20 razy więcej), używa Aggregate i Index Scan
- Koszt jest dużo większy w zapytaniu bez funkcji okna

SQLite

- Zapytanie z funkcją okna używa Full Scan tabeli product_history
- Zapytanie bez funkcji okna wykonuje Full Scan tabeli dwa razy

Zadanie 8

Wykonaj kilka "własnych" przykładowych analiz.

Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)? Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite

Średnia ruchoma (moving average) dla wartości sprzedaży produktu w danym dniu, z oknem 3 dniowym:

```
select  
  productid,
```

```

date,
value * quantity as sales_value,

avg(value * quantity) over (
  partition by productid
  order by date
  rows between 2 preceding and current row
) as moving_avg
from product_history;

```

Rezultat wywołania powyższego zapytania:

The screenshot shows a SQL playground interface. The top panel contains a query:


```

with base as (
  select
    productid,
    date,
    value * quantity as sales_value
  from product_history
)
select
  
```

 The bottom panel displays the results of the query in a table format. The table has five columns: productid, date, sales_value, and moving_avg. The results show data for productid 1 from 1940-01-02 to 1940-01-16. The moving_avg column shows a trend that starts at 1585 and ends at 2153.33.

productid	date	sales_value	moving_avg
1	1940-01-02	1585	1585
1	1940-01-03	2567	2076
1	1940-01-04	2742	2298
1	1940-01-05	1403	2237.33
1	1940-01-06	2658	2267.67
1	1940-01-07	2025	2028.67
1	1940-01-08	2123	2268.67
1	1940-01-09	1874	2007.33
1	1940-01-10	2331	2109.33
1	1940-01-11	2102	2102.33
1	1940-01-12	1787	2073.33
1	1940-01-13	1255	1714.67
1	1940-01-14	1772	1604.67
1	1940-01-15	1080	1669
1	1940-01-16		2153.33

Użycie funkcji `percent_rank()`, procentowa pozycja produktu w kategorii względem ceny

```

select
  productid,
  productname,
  categoryid,
  unitprice,

  percent_rank() over (
    partition by categoryid
    order by unitprice
  ) as price_percent_rank
from products;

```

Rezultat wywołania powyższego zapytania:

The screenshot shows a SQL playground with a query editor and a results pane. The query is as follows:

```

1 select
2   productid,
3   productname,
4   categoryid,
5   unitprice,
6
7   round(
8     percent_rank() over (
9       partition by categoryid
10      order by unitprice

```

The results pane displays a table with 77 rows. The columns are ProductID, ProductName, CategoryID, UnitPrice, price_percent_rank, and price_category. The data is sorted by CategoryID and then by UnitPrice within each category.

ProductID	ProductName	CategoryID	UnitPrice	price_percent_rank	price_category
24	Guaraná Fantástica	1	4.5	0	cheap
75	Rhönbräu Klosterbier	1	7.75	0.09	cheap
34	Sasquatch Ale	1	14	0.18	cheap
67	Laughing Lumberjack Lager	1	14	0.18	cheap
70	Outback Lager	1	15	0.36	average
1	Chai	1	18	0.45	average
35	Steeleye Stout	1	18	0.45	average
39	Chartreuse verte	1	18	0.45	average
76	Lakkalikööri	1	18	0.45	average
2	Chang	1	19	0.82	expensive
43	Ipoh Coffee	1	46	0.91	expensive
38	Côte de Blaye	1	263.5	1	expensive
3	Aniseed Syrup	2	10	0	cheap
77	Original Frankfurter grüne Soße	2	13	0.09	cheap

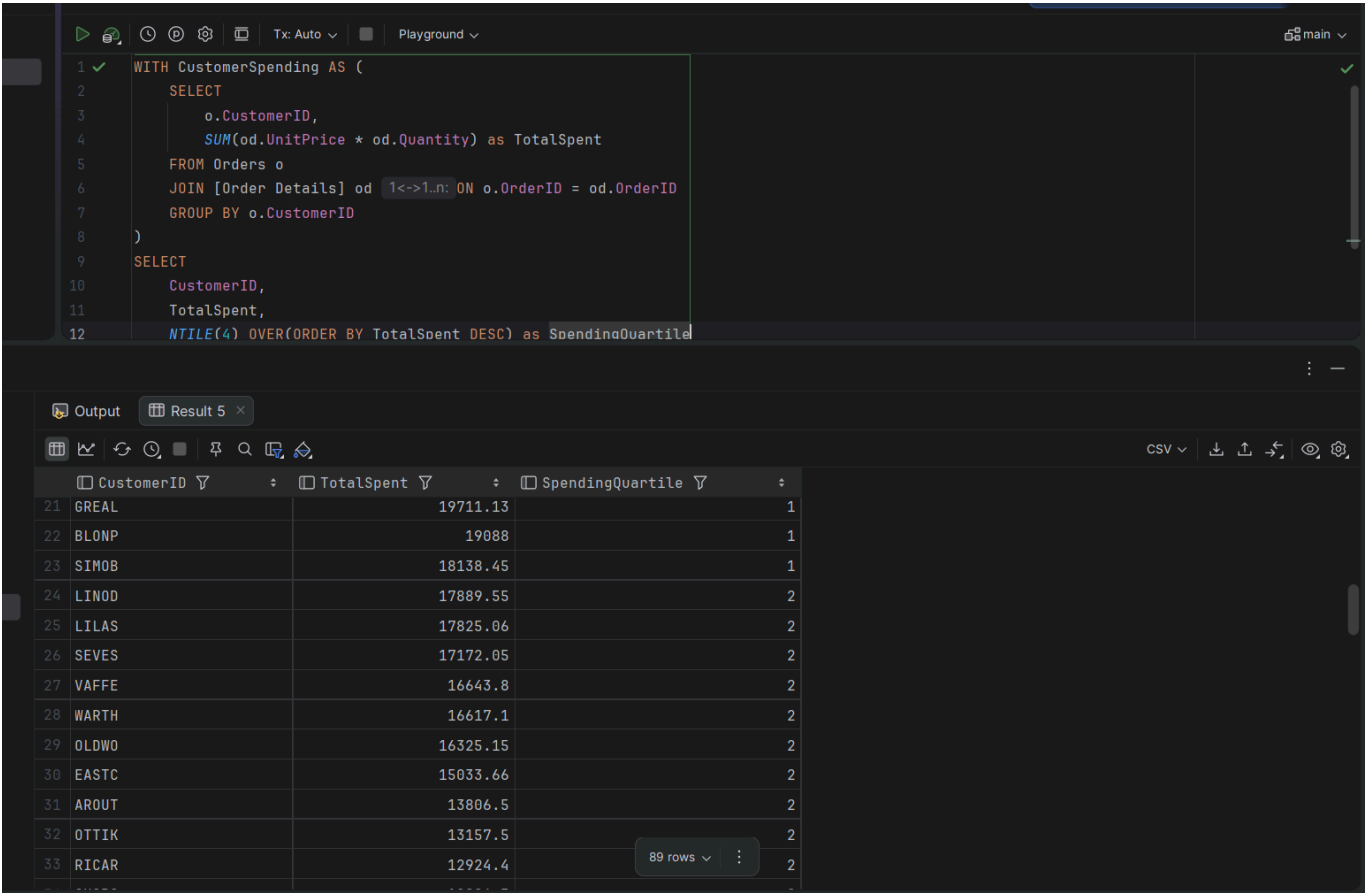
Ranking klientów na podstawie łącznej wartości zamówień, z podziałem na kwartyle z wykorzystaniem funkcji `ntile()`:

```

WITH CustomerSpending AS (
  SELECT
    o.CustomerID,
    SUM(od.UnitPrice * od.Quantity) as TotalSpent
  FROM Orders o
  JOIN [Order Details] od ON o.OrderID = od.OrderID
  GROUP BY o.CustomerID
)
SELECT
  CustomerID,
  TotalSpent,
  NTILE(4) OVER(ORDER BY TotalSpent DESC) as SpendingQuartile
FROM CustomerSpending;

```

Rezultat wywołania powyższego zapytania:



Punktacja

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2
razem	11